

OSE AUDITOR

Official Positioning & Product Strategy Document (Version 1.0)

Parent Company

Crestsek Technology Ltd

1. MISSION

OSE exists to prevent financial loss caused by software logic flaws before deployment.

Modern developers increasingly rely on AI-assisted coding tools to write software at unprecedented speed. While these tools accelerate development, they also introduce subtle vulnerabilities, broken business logic, race conditions, authorization failures, and transactional flaws that traditional code review processes often miss.

OSE was created to serve as a dedicated security and logic verification layer focused on detecting vulnerabilities capable of causing real-world financial damage.

2. WHAT OSE IS

OSE is an autonomous financial and logic exploit detection engine that uses deterministic code analysis and AI remediation to discover money-losing vulnerabilities before deployment.

OSE analyzes code structure, execution flow, authorization paths, transactional operations, and financial state transitions to identify vulnerabilities that may result in:

- Financial loss
- Unauthorized access
- Payment manipulation
- Double spending
- Logic exploitation
- Data leakage
- State corruption

After detection, OSE generates precise remediation guidance and secure replacement code.

3. WHAT OSE IS NOT

OSE is not:

- A chatbot
- A code generation assistant
- A syntax formatter
- A code beautifier
- A generic AI wrapper
- A traditional static scanner
- A replacement for software engineers

OSE exists for one purpose:

To identify exploitable financial and logic vulnerabilities before software reaches production.

4. CORE PHILOSOPHY

General AI models are optimized to write code.

OSE is optimized to distrust code.

General AI systems operate as builders.

OSE operates as an adversarial reviewer.

Its default assumption is:

"The code is dangerous until proven safe."

5. PRIMARY DIFFERENTIATOR

Traditional LLMs evaluate source code as text.

OSE evaluates software as execution logic.

Before any AI remediation occurs, OSE performs deterministic analysis to understand:

- Data flow
- State transitions
- Financial operations

- Authorization chains
- Execution ordering
- Risk boundaries

The AI component is used primarily for remediation and explanation.

Detection remains deterministic.

6. INITIAL MARKET FOCUS

OSE will launch by focusing exclusively on Web2 backend systems.

Initial supported ecosystems:

- JavaScript
- TypeScript
- Node.js

Initial vulnerability focus:

- Payment flows
- Transaction handling
- Authentication failures
- Authorization bypasses
- Race conditions
- Database integrity violations
- Secret exposure
- Financial workflow manipulation

This phase serves as OSE's proving ground.

7. LONG-TERM PRODUCT EXPANSION

After Web2 stabilization:

Phase 2: Web3

Supported Languages:

- Solidity

Focus Areas:

- Reentrancy
- Flash-loan exploits
- Ownership control failures
- Asset drainage paths
- State manipulation

Phase 3: Quant Systems

Supported Languages:

- Python
- MQL4
- MQL5

Focus Areas:

- Risk management failures
- Slippage protection
- Spread protection
- Position sizing errors
- Trading execution flaws

Web2 funds Web3.

Web3 funds Quant.

8. PRODUCT ARCHITECTURE

OSE uses a Hybrid Local + Cloud Architecture.

Local Engine

Responsibilities:

- Project scanning
- AST generation
- File filtering
- Risk extraction
- Context compression

Benefits:

- Privacy

- Reduced token consumption
- Lower infrastructure costs
- Faster analysis

Cloud Platform

Responsibilities:

- Credit enforcement
 - User management
 - Premium remediation
 - AI orchestration
 - Rule updates
 - Subscription handling
-

9. THE FINANCIAL SEMANTIC ANALYZER (FSA)

The Financial Semantic Analyzer is the proprietary core of OSE.

Responsibilities:

1. Financial Symbol Detection
2. State Transition Mapping
3. Authorization Analysis
4. Business Logic Verification
5. Exploit Pattern Detection
6. Risk Classification

The FSA represents OSE's primary intellectual property.

AI generates fixes.

FSA finds the vulnerabilities.

10. OPEN SOURCE STRATEGY

OSE follows an Open Core model.

Open Source Components

Public:

- Local CLI
- Parser
- File filtering engine
- Community rules
- Documentation

Purpose:

- Trust
- Adoption
- Community contribution
- Transparency

Proprietary Components

Private:

- Premium remediation engine
- Credit infrastructure
- User ledger
- Prompt systems
- Cloud orchestration
- Subscription systems

Purpose:

- Revenue generation
- Platform sustainability
- Competitive advantage

11. IDEAL CUSTOMER PROFILE

Primary Users:

- Solo developers
- Startup engineers
- Technical founders
- Freelance backend contractors
- AI-native developers

Characteristics:

- Ship software rapidly
- Use Cursor, Aider, Cline, Claude Code
- Handle transactional systems
- Lack dedicated security teams

Pain Point:

"I can build faster than ever, but I no longer trust everything AI writes."

OSE solves that problem.

12. GO-TO-MARKET STRATEGY

Stage 1: Founder-led outreach.

Target:

- Cursor communities
- Reddit developers
- Node.js developers
- Backend engineers

Goal:

First 10 users.

Stage 2: GitHub proof distribution.

Method:

- Find real vulnerabilities
- Submit fixes
- Demonstrate value publicly

Goal:

First 100 users.

Stage 3: MCP ecosystem dominance.

Method:

- Registry listings
- Open-source adoption
- Tool ecosystem integrations

Goal:

1,000+ users.

Stage 4: Invite-only growth.

Method:

- Referral tokens
- Controlled access
- Community prestige

Goal:

10,000+ users.

13. CONTENT STRATEGY

OSE does not publish generic motivational content.

OSE publishes evidence.

Primary content:

- Exploit breakdowns
- Security case studies
- Vulnerability comparisons
- Real code audits
- Benchmark results

Core message:

"Look what OSE caught."

14. COMPETITIVE POSITIONING

Against AI Coding Assistants:

They write code.

OSE audits code.

Against Traditional Security Scanners:

They identify issues.

OSE identifies issues and generates actionable remediation.

Against Enterprise Security Platforms:

They focus on large corporate workflows.

OSE integrates directly into the developer workflow.

15. SUCCESS METRIC

OSE succeeds when developers begin saying:

"I don't deploy AI-generated code until OSE reviews it."

That is the long-term position OSE seeks to own.

Not another coding assistant.

Not another scanner.

The final security review layer between AI-generated software and production deployment.

FOUNDING PRINCIPLE

Build fast.

Trust nothing.

Audit everything.